

Exceptions

A general mechanism for handling errors.

- Automatically used by Python for various types of errors that may occur while the program is running.
- Can be used by the programmer to handle errors or "unexpected events" that the program itself detects.

Example

This is what it looks like if we divide by 0 in the Python console:

```
>>> 1/0
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
ZeroDivisionError: division by zero
```

Other examples of possible exceptions:

`math.sin('a')` creates a `TypeError`

`math.log(-1)` creates a `ValueError`

Exceptions can be "caught"

```
from math import sqrt

while True:
    x = input("Give a positive number: ")
    try:
        y = sqrt(int(x))
    except ValueError:
        print(f"Squarehead! {x} is not a positive number!")
    else:
        print(f"The square root of {x} is {y}")
```



Test run

```
Give a positive number: 2
The square root of 2 is 1.4142135623730951
Give a positive number: -1
Squarehead! -1 is not a positive number!
Give a positive number: 4
The square root of 4 is 2.0
Give a positive number: a
Squarehead! a is not a positive number!
Give a positive number: 9
The square root of 9 is 3.0
Give a positive number:
```



Example

```
from math import sqrt, log

def compute(x):
    return sqrt(x) + log(2-x) + 1/x

while True:
    x = input('Give a value: ')
    try:
        y = compute(int(x))
    except (ValueError,
            ZeroDivisionError) as e:
        print('*** Error:', e)
    else:
        print('Result: ', y)
```



Test run

```
Give a value: 1
Result: 2.0
Give a value: 2
*** Error: math domain error
Give a value: -2
*** Error: math domain error
Give a value: a
*** Error: invalid literal for int() with base 10: 'a'
Give a value: 0
*** Error: division by zero
Give a value:
```

Handling different exceptions differently

```
from math import sqrt, log

def compute(x):
    return sqrt(x) + log(2-x) + 1/x

while True:
    x = input('Give a value: ')
    try:
        y = compute(int(x))
    except ValueError as e:
        print('*** Illegal argument!', e)
    except ZeroDivisionError as e:
        print('*** Division by zero!', e)
    else:
        print('Result: ', y)
```



Writing exception classes

```
class MyException(Exception):
    def __init__(self, arg):
        self.arg = arg
        super().__init__(self.arg)

def compute(x):
    result= sqrt(x) + log(2-x) + 1/x
    if result > 1:
        raise MyException('Too big')
    return result

while True:
    x = input('Give a value: ')
    try:
        y = compute(int(x))
    except (ValueError, ZeroDivisionError, MyException) as e:
        print(e)
    else:
        print('Result: ', y)
```



UPPSALA
UNIVERSITET

The end





UPPSALA
UNIVERSITET





UPPSALA
UNIVERSITET





UPPSALA
UNIVERSITET





UPPSALA
UNIVERSITET



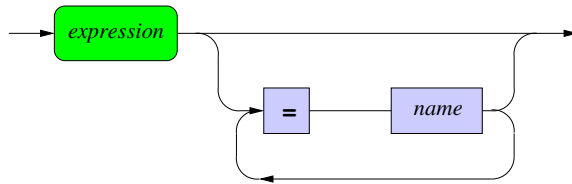
How to start?

- Read the MA2 text
- Download the the files `MA2.py` and `MA2tokenizer.py`
- Also download the the `MA2test.txt` but it is of no use until you have implemented the `file` command.
- Run the `MA2.py` and check that addition, multiplication and parenthesis works.
- Modify the code so that subtraction and division are handled.
- ...

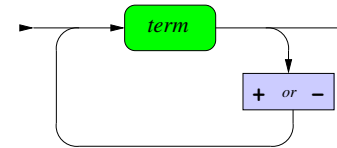


Syntax

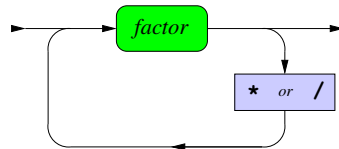
assignment



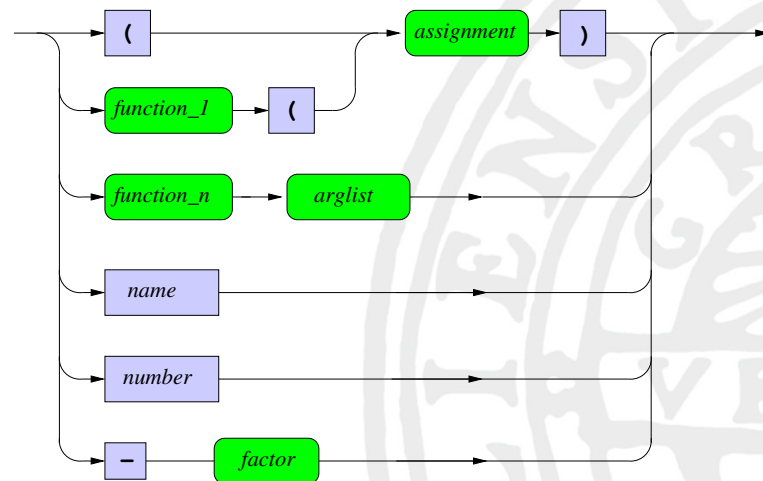
expression



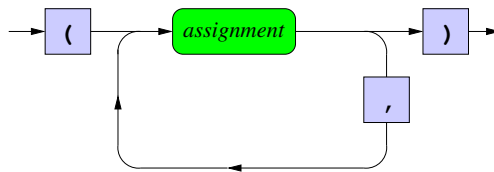
term



factor



arglist





UPPSALA
UNIVERSITET

The end

I will record another lecture where I discuss

- Error handling with exceptions
- Function objects
- And more details.

